

# CloudShield AI / DAIL

## 10 — LLM Integration Specification

Version 1.0 | Final LLM Integration Baseline | 1 September 2026

Purpose: Define the exact interface between DAIL and an LLM used to generate infrastructure repair candidates. This specification constrains context construction, prompt contracts, model configuration, output parsing, patch extraction, validation, retries, token/context budgeting, failure handling, observability, and the strict boundary preventing the LLM from becoming a source of trust or a promotion authority.

### Document Control

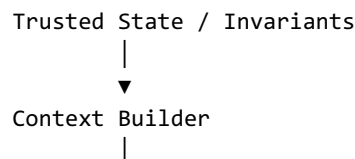
| Field            | Value                                                                                                                                                             |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document ID      | CSAI-DAIL-LLM-010                                                                                                                                                 |
| Status           | FINAL LLM INTEGRATION BASELINE                                                                                                                                    |
| Parent documents | 01 SRS; 02 Architecture; 03 Functional; 04 Terraform/AWS; 05 Domain/Data; 06 Lifecycle; 07 Identity/Dependency; 08 Impact/Invalidation; 09 Verification/Promotion |
| Primary output   | Candidate Patch / Candidate State proposal                                                                                                                        |
| LLM role         | Generate repair proposals only                                                                                                                                    |
| LLM authority    | No authority over verification, invariant truth, or promotion                                                                                                     |
| Core principle   | LLM output is untrusted input and must pass deterministic gates.                                                                                                  |

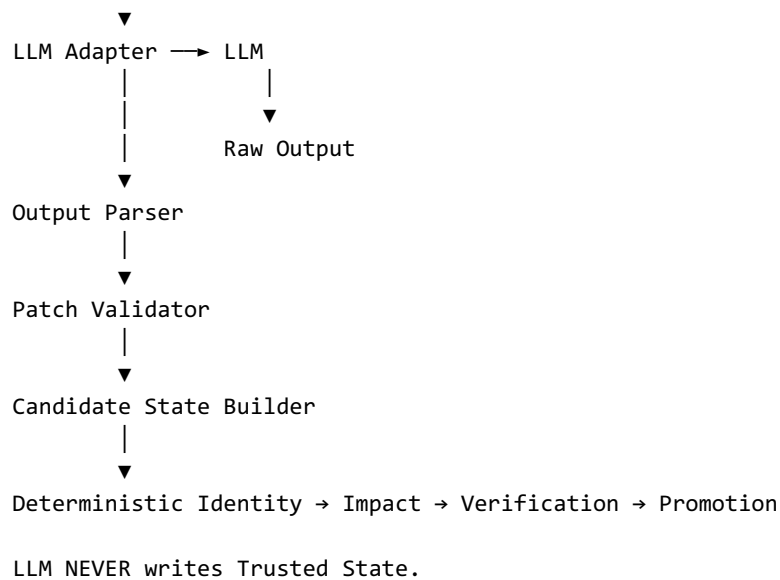
### 1. Scope

This document owns the LLM adapter and candidate-generation workflow. It does not define the underlying model provider's proprietary behavior and does not permit model output to alter Trusted State directly.

- Prompt/context construction.
- Model configuration.
- Structured output contract.
- Patch extraction and validation.
- Context/token budgeting.
- Retry policy.
- Generation failure handling.
- LLM observability and reproducibility.
- LLM-to-candidate handoff.

### 2. Trust Boundary





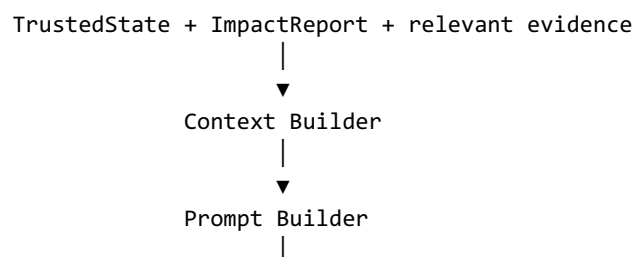
### 3. LLM Design Principles

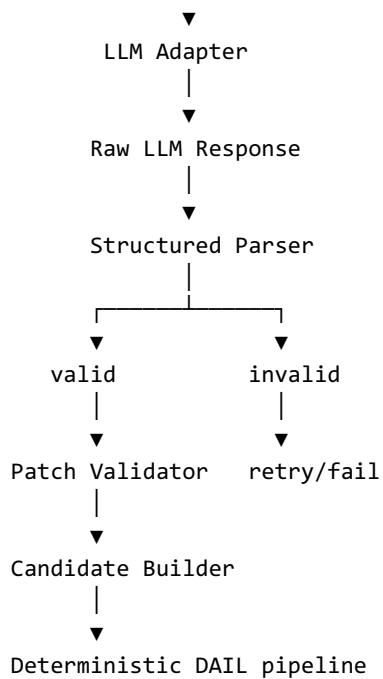
- Treat every model response as untrusted.
- Provide the minimum sufficient context required for repair generation.
- Prefer structured output over free-form extraction.
- Keep system/developer instructions deterministic and versioned.
- Record model and prompt metadata required for reproducibility.
- Do not ask the LLM to decide whether a candidate is safe.
- Do not allow model confidence to satisfy verification.
- Do not allow the model to mutate state outside the candidate-patch channel.
- Use deterministic validators before any candidate enters DAIL analysis.

### 4. LLM Responsibilities

| Allowed                                         | Forbidden                                         |
|-------------------------------------------------|---------------------------------------------------|
| Propose Terraform changes.                      | Declare a candidate safe.                         |
| Explain a proposed patch.                       | Modify Trusted State.                             |
| Suggest minimal repair actions.                 | Override verifier failures.                       |
| Return structured patch output.                 | Mark invariants PROTECTED.                        |
| Use supplied state/context.                     | Invent unsupported infrastructure facts as proof. |
| Generate alternative candidates when requested. | Authorize promotion.                              |

### 5. Candidate Generation Pipeline





## 6. Context Contract

The context builder must construct a bounded, explicit representation of the repair problem.

```
LLMContext {  
    context_version  
    trusted_state_hash  
    relevant_resources[]  
    relevant_invariants[]  
    impact_summary  
    dependency_summary  
    allowed_resource_types[]  
    allowed_change_scope[]  
    known_constraints[]  
    unsupported_constraints[]  
    patch_format  
}
```

- Include only resources relevant to the requested repair where possible.
- Include protected invariants that constrain the repair.
- Include impact reasons and dependency paths when they materially affect repair generation.
- Explicitly distinguish known facts from unknown/unsupported facts.

## 7. Context Minimization

The LLM should not receive the entire environment when a smaller semantically sufficient context can solve the repair. Context reduction improves token efficiency and reduces opportunities for irrelevant generation.

- Prefer affected resource/invariant neighborhoods.
- Include parent-state facts necessary to avoid destructive changes.

- Include canonical Terraform snippets for relevant resources.
- Include relevant dependency relationships.
- Include required functional/security constraints.
- Exclude unrelated historical attempts unless explicitly needed for the experiment.

## 8. Context Safety

- Secrets, credentials, private keys, tokens, and unnecessary sensitive values must be removed before prompt construction.
- Runtime values that are not required for generation should be omitted.
- Unknown values must be represented explicitly rather than fabricated.
- Prompt context must identify candidate-generation boundaries.

## 9. Prompt Contract

System/Developer layer:

- role
- hard constraints
- output schema
- prohibited actions
- supported Terraform/AWS model
- safety boundary

Task layer:

- current state
- requested repair objective
- protected invariants
- relevant context
- required output

Prompt templates are versioned artifacts. A prompt change that could affect experiment behavior requires a new prompt version.

## 10. Required Generation Instructions

- Generate only a Terraform patch within the supplied supported model.
- Preserve unrelated resources unless explicitly required.
- Make the smallest sufficient change where possible.
- Do not claim runtime verification.
- Do not claim promotion or trust.
- Do not invent provider IDs or runtime facts.
- Return machine-readable output.
- If the problem cannot be safely solved with the supplied context, return an explicit inability/uncertainty signal rather than fabricating facts.

## 11. Structured Output Contract

```
LLMRepairResponse {
  schema_version
```

```

    status = PATCH | NO_SOLUTION | UNCERTAIN | INVALID
    summary
    patch
    changed_addresses[]
    assumptions[]
    rationale
  }

```

| Field             | Requirement                                            |
|-------------------|--------------------------------------------------------|
| status            | Required enum.                                         |
| summary           | Short explanation of proposed repair.                  |
| patch             | Terraform HCL patch or null depending on status.       |
| changed_addresses | Addresses the model intends to modify.                 |
| assumptions       | Explicit assumptions; never treated as verified facts. |
| rationale         | Generation explanation; not a safety proof.            |

## 12. Patch Contract

The patch is the only model output that may enter the Candidate State construction path.

```

Patch {
  patch_id
  source = LLM
  content
  content_hash
  parent_state_id
  llm_request_id
}

```

- Patch content must be immutable once accepted into the attempt record.
- Patch hash is deterministic.
- Patch must reference exactly one parent Trusted State.
- The parser must reject missing or ambiguous patch content.

## 13. Patch Validation Pipeline

1. Validate structured response schema.
2. Confirm status is PATCH.
3. Extract patch content.
4. Validate HCL syntax.
5. Check allowed resource types/constructs.
6. Reject forbidden operations/constructs.
7. Resolve references within supported model.
8. Normalize patch using Document 04 semantics.
9. Construct Candidate State.
10. Run deterministic Identity/Impact/Verification.

LLM output is never executed against production infrastructure as part of candidate generation.

## 14. Terraform Safety Restrictions

- Only resource types supported by Document 04 may be modified/created.
- Unsupported dynamic constructs are rejected or marked unsupported.
- Credentials/provider secrets may not be introduced by generated patches.
- Destructive operations outside the defined experiment scope are rejected.
- Provider/runtime IDs must not be invented.
- Changes outside the allowed scope should fail validation unless explicitly permitted.
- Generated patches must not modify the experiment harness, verifier, promotion policy, or evidence system.

## 15. Allowed Change Scope

```
AllowedChangeScope {
  resource_addresses[]
  resource_types[]
  attributes[]
  operation_types = CREATE | MODIFY | DELETE | REPLACE
}
```

The scope is generated by the experiment/controller, not chosen by the LLM.

## 16. Model Configuration

```
LLMConfig {
  provider
  model_name
  model_version | deployment
  temperature
  top_p | equivalent
  max_output_tokens
  timeout
  retry_policy_id
  prompt_version
  response_schema_version
}
```

All fields that can influence generation must be recorded for reproducibility.

## 17. Determinism and Sampling

The project may intentionally evaluate deterministic and stochastic generation conditions. Regardless of sampling mode, the exact configuration must be persisted.

| Condition               | Requirement                                                                            |
|-------------------------|----------------------------------------------------------------------------------------|
| Deterministic condition | Use fixed/low-variance settings supported by the provider; record exact configuration. |
| Stochastic condition    | Record sampling parameters and randomization metadata where available.                 |
| Repeated attempt        | Each attempt gets its own request/response record.                                     |

The experiment protocol, not the LLM adapter, determines whether a condition is considered deterministic or stochastic.

## 18. Token and Context Budgeting

Token budgeting is a resource constraint, not a safety constraint. Truncating required context must not silently occur.

Budget:

```
max_context_tokens
reserved_output_tokens
safety_margin
estimated_input_tokens
estimated_output_tokens
```

- If context exceeds the configured budget, apply deterministic context reduction.
- Prioritize directly affected resources/invariants and required constraints.
- Record what was omitted.
- If required information cannot fit, return `CONTEXT_INSUFFICIENT` rather than fabricate a repair.

## 19. Context Reduction Priority

| Priority | Context                                                    |
|----------|------------------------------------------------------------|
| 1        | Repair objective and blocking invariants.                  |
| 2        | Affected resources and exact relevant attributes.          |
| 3        | Relevant dependency paths.                                 |
| 4        | Required Terraform/AWS semantic constraints.               |
| 5        | Parent-state information necessary to preserve invariants. |
| 6        | Supporting/non-critical explanatory context.               |
| 7        | Historical/contextual information not required for repair. |

## 20. LLM Request Model

```
LLMRequest {
  request_id
  attempt_id
  parent_state_id
  context_hash
  prompt_hash
  prompt_version
  model_config
  created_at
}
```

Raw prompt content may be redacted where necessary, but hashes and sufficient metadata must remain for experiment correlation.

## 21. LLM Response Model

```
LLMResponse {
  response_id
  request_id
  raw_response
  parsed_status
}
```

```

    extracted_patch
    response_hash
    parse_errors[]
    created_at
}

```

## 22. Parsing Failure

| Failure                                 | Action                                                           |
|-----------------------------------------|------------------------------------------------------------------|
| Malformed JSON/schema                   | INVALID; retry if policy permits.                                |
| Missing patch                           | NO_SOLUTION or INVALID.                                          |
| Invalid HCL                             | Reject candidate; optionally retry.                              |
| Unsupported resource                    | Reject/UNSUPPORTED.                                              |
| Forbidden operation                     | Reject.                                                          |
| Ambiguous patch boundaries              | Reject; never guess.                                             |
| Extraneous text in structured-only mode | Reject or parse only under explicitly versioned tolerant parser. |

## 23. Retry Policy

Retries are bounded and evidence-preserving.

```

RetryPolicy {
    max_attempts
    retry_on = [TRANSIENT_ERROR, INVALID_OUTPUT, CONTEXT_INSUFFICIENT, NO_SOLUTION]
    backoff
    prompt_adjustment_policy
}

```

- Never retry indefinitely.
- Every retry is a distinct request/response record.
- Do not silently change the parent Trusted State.
- Do not alter safety constraints merely to obtain a patch.
- A retry may adjust context formatting or request clarification, but must remain within the same experiment policy.

## 24. LLM Failure Classes

| Class                | Meaning                                                   |
|----------------------|-----------------------------------------------------------|
| TRANSIENT_ERROR      | Provider/network/temporary service failure.               |
| TIMEOUT              | Model did not respond within configured limit.            |
| RATE_LIMIT           | Provider refused due to quota/rate policy.                |
| INVALID_OUTPUT       | Response cannot satisfy the output contract.              |
| NO_SOLUTION          | Model explicitly cannot propose a valid repair.           |
| CONTEXT_INSUFFICIENT | Required repair information cannot fit or is unavailable. |
| POLICY_VIOLATION     | Generated output violates DAIL generation constraints.    |
| UNSUPPORTED          | Requested repair requires unsupported semantics.          |

## 25. LLM Confidence

Any confidence score returned by the model is advisory metadata only.



```
confidence = metadata
confidence ≠ verification
confidence ≠ evidence
confidence ≠ promotion authority
```

## 26. Rationale Handling

The model's rationale may help explain why a patch was proposed, but it cannot establish infrastructure facts that are not independently verified.

- Store rationale as generation evidence.
- Never use rationale as proof of connectivity/security.
- Never let rationale override deterministic verifier output.
- Clearly label assumptions as assumptions.

## 27. Candidate Isolation

- Each LLM-generated patch is isolated as a Candidate State proposal.
- LLM output cannot mutate Trusted State.
- LLM output cannot modify invariant definitions.
- LLM output cannot modify verifier code/configuration.
- LLM output cannot modify promotion policy.
- LLM output cannot write directly to cloud infrastructure.

## 28. Prompt Injection / Untrusted Context

Terraform strings, resource names, comments, descriptions, or external text may contain instructions that look like prompts. They are data, not authority.

- Treat all infrastructure content as untrusted data.
- Delimit infrastructure data from control instructions.
- Do not allow resource descriptions/comments to override system constraints.
- Do not allow generated output to redefine its own schema or permissions.

## 29. Tool Use Boundary

If the LLM integration supports tools, tool permissions must be narrowly scoped.

| Tool                           | Policy                                                                                  |
|--------------------------------|-----------------------------------------------------------------------------------------|
| Read normalized state          | Allowed.                                                                                |
| Read approved fixture/context  | Allowed.                                                                                |
| Write Candidate Patch artifact | Allowed through adapter boundary.                                                       |
| Run verifier                   | Optional only through controlled interface; result remains external evidence.           |
| Modify Trusted State           | Forbidden.                                                                              |
| Modify promotion policy        | Forbidden.                                                                              |
| Apply Terraform to production  | Forbidden.                                                                              |
| Access secrets                 | Forbidden unless an explicitly isolated future experiment requires it; not part of MVP. |

## 30. Model Provider Abstraction

The application should expose a provider-neutral adapter.

```
LLMProvider:  
    generate(request: LLMRequest) -> LLMResponse
```

- Provider-specific APIs stay inside adapter implementations.
- Domain objects remain provider-neutral.
- Model metadata is persisted in LLMRequest.
- Provider failures map to normalized failure classes.

## 31. LLM Adapter Contract

```
generate(context, config):  
    validate config  
    estimate budget  
    build versioned prompt  
    record request metadata  
    call provider  
    record raw response safely  
    parse structured response  
    validate patch  
    return normalized LLMResponse
```

## 32. Observability

- Request ID and attempt ID.
- Model/provider/deployment.
- Prompt version/hash.
- Context version/hash.
- Response hash.
- Latency.
- Token usage where available.
- Retry count.
- Failure class.
- Parser/validator version.
- Final generation status.

## 33. Privacy and Secret Handling

- Redact cloud credentials and secrets.
- Do not place private keys in prompts.
- Minimize sensitive infrastructure values.
- Do not persist unnecessary raw provider metadata.
- Define access controls around raw prompts/responses where required.
- Log hashes/metadata when raw content should not be retained.

## 34. Reproducibility

A generation attempt should be reconstructable to the extent permitted by the provider.

- Persist model/deployment identifier.
- Persist prompt/context versions and hashes.
- Persist generation parameters.
- Persist patch content/hash.
- Persist parser and validator versions.
- Persist fixture/state hashes.
- Persist timestamps and request/response IDs.

Exact byte-for-byte reproduction may not be possible for proprietary stochastic providers; the record must still identify the exact experimental condition.

## 35. Candidate Generation Scenarios

| Scenario                        | Expected output                                         |
|---------------------------------|---------------------------------------------------------|
| Simple SG rule repair           | PATCH with minimal HCL change.                          |
| Ambiguous context               | UNCERTAIN or CONTEXT_INSUFFICIENT; no fabricated patch. |
| Unsupported resource dependency | UNSUPPORTED/NO_SOLUTION.                                |
| Attempt to change verifier      | POLICY_VIOLATION.                                       |
| Attempt to apply infrastructure | Rejected by adapter/validator.                          |
| No valid repair found           | NO_SOLUTION; candidate not created.                     |

## 36. Canonical Security Repair Prompt Objective

The LLM may be asked to remove public SSH exposure while preserving the protected functional invariant. The model must receive the relevant SG/routing/EC2/RDS context and be instructed that safety is determined later by deterministic verification.

Objective:

Propose the smallest supported Terraform change that removes the forbidden public SSH rule while preserving the declared EC2→RDS TCP/5432 requirement.

Output:

structured PATCH response only.

## 37. Canonical Functional Preservation Objective

When generating a security repair, the prompt should explicitly state functional constraints that the patch must preserve, but the LLM's assertion that they are preserved is never accepted as proof.

Constraint:

Do not break INV-FUNC-001.

Proof:

Verification Engine only.

## 38. Anti-Shortcut Rules

- No 'looks safe' promotion.
- No LLM self-certification.
- No rationale-as-evidence.
- No confidence-as-proof.
- No automatic acceptance of model-supplied resource identity.
- No automatic acceptance of model-supplied dependency claims.
- No skipping verification because the model predicts PASS.

## 39. Testing Strategy

### 39.1 Adapter Tests

- Provider request mapping.
- Timeout/rate-limit mapping.
- Response normalization.
- Metadata capture.
- Retry behavior.

### 39.2 Parser Tests

- Valid structured output.
- Malformed output.
- Missing patch.
- Invalid status.
- Extra fields.
- Ambiguous patch.
- Invalid HCL.

### 39.3 Security Tests

- Prompt injection in resource descriptions.
- Attempt to modify verifier/policy.
- Attempt to introduce credentials.
- Attempt to use unsupported resource.
- Oversized context.

### 39.4 Reproducibility Tests

- Same context/config produces identical request metadata.
- Patch hash is stable.
- Prompt/context versions are persisted.

## 40. Acceptance Criteria

- LLM output can enter the system only through the normalized candidate-generation boundary.
- All generation conditions are versioned and recorded.
- Structured responses are validated before patch construction.
- Unsupported/forbidden changes are rejected.
- Context overflow never silently drops required safety constraints.

- Retries are bounded and auditable.
- LLM confidence/rationale cannot satisfy verification.
- LLM cannot mutate Trusted State or promotion policy.
- Secrets are removed from prompts and persistent artifacts.
- Every accepted patch has a parent Trusted State and immutable hash.
- Deterministic DAIL verification remains mandatory after generation.

## 41. Non-Goals

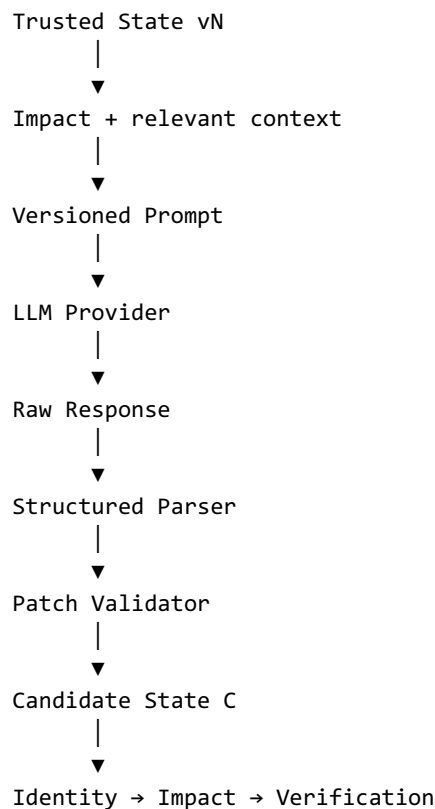
- Selecting the best LLM provider.
- Defining proprietary provider internals.
- Using an LLM as a verifier of its own output.
- Allowing autonomous production deployment.
- Replacing deterministic infrastructure analysis with natural-language reasoning.

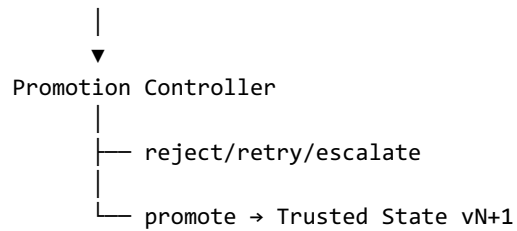
## 42. Final LLM Integration Directive

The LLM is a candidate generator, not a safety oracle. DAIL shall treat model output as untrusted input, constrain it through a versioned context/prompt contract, validate it deterministically, isolate it as a Candidate State, and subject it to the same Identity → Impact → Verification → Promotion pipeline as every other candidate.

No model-generated statement, confidence score, rationale, or proposed dependency can independently establish trust. The only path from LLM output to Trusted State is through deterministic validation, sufficient verification evidence, and the Promotion Controller.

## Appendix A — End-to-End LLM Flow





## Appendix B — Data Dependencies

| Object             | Source                                                             |
|--------------------|--------------------------------------------------------------------|
| LLMContext         | TrustedState + ImpactReport + DependencyGraph + Invariant Registry |
| LLMRequest         | LLMContext + LLMConfig                                             |
| LLMResponse        | LLM provider                                                       |
| Patch              | Parsed LLMResponse                                                 |
| CandidateState     | Patch + parent TrustedState                                        |
| VerificationReport | Candidate + Verification Work Set                                  |
| PromotionDecision  | VerificationReport + policy                                        |

## Appendix C — Next Specification

Document 11 — Evidence & Logging Specification defines the immutable evidence/event model, logging taxonomy, correlation IDs, redaction rules, audit trail, artifact storage, and reproducibility requirements across the entire DAIL pipeline.